

Introduction to Pandas

DataFrame, Series, Data Cleaning, Grouping, and Business Data Analysis

Duc-Minh Vu

SLSCM Lab - FDA - NEU

August 17, 2026

Learning Outcomes

By the end of this lesson, learners will be able to:

- ▶ Explain the role of Pandas in data science and business analytics.
- ▶ Distinguish between `Series` and `DataFrame`.
- ▶ Create, inspect, select, filter, and sort tabular data.
- ▶ Read and write CSV, Excel, JSON, and text-based datasets.
- ▶ Detect and handle missing values, duplicate rows, and inconsistent data types.
- ▶ Transform columns and apply simple functions.
- ▶ Use `groupby()`, `agg()`, `merge()`, `concat()`, and pivot tables.
- ▶ Compute descriptive statistics, correlation, and simple time-series summaries.

Lesson Roadmap

- 1 Why learn Pandas?
- 2 What Pandas is
- 3 Pandas in the Data Science workflow
- 4 Environment setup
- 5 Your first DataFrame
- 6 Series and DataFrame
- 7 Reading Pandas commands
- 8 Inspecting, selecting, filtering, and sorting
- 9 Reading and writing data files
- 10 Data cleaning
- 11 Columns, grouping, and aggregation
- 12 Merge, concat, and pivot tables
- 13 Correlation, visualization, time series
- 14 Practice exercises

Why Learn Pandas?

Pandas is a practical bridge between raw files and data-science models.

- ▶ Real datasets are often messy, incomplete, and stored in different formats.
- ▶ Pandas helps organize records into rows and attributes into columns.
- ▶ Pandas supports inspection, cleaning, transformation, aggregation, and reporting.
- ▶ Pandas works well with NumPy, Matplotlib, Scikit-learn, and Statsmodels.

Business analytics workflow

Raw data → clean table → summary insight → model-ready dataset.

What Is Pandas?

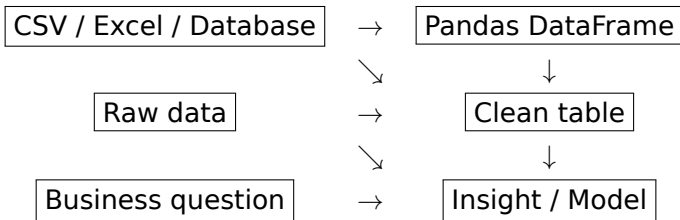
Pandas is an open-source Python library for **data manipulation and analysis**. It is built on top of NumPy and provides high-level tools for structured and tabular data.

Pandas is especially useful for:

- ▶ reading data from CSV, Excel, JSON, and text files;
- ▶ cleaning, transforming, and preparing datasets;
- ▶ selecting, filtering, grouping, and aggregating observations;
- ▶ combining multiple datasets;
- ▶ computing descriptive statistics and quick visualizations.

Pandas in the Data Science Workflow

Pandas is usually used to turn raw data into a clean, analyzable table.



- ▶ NumPy supports efficient numerical computing.
- ▶ Pandas organizes data into rows, columns, and indexes.
- ▶ Matplotlib/Seaborn visualize results from DataFrames.
- ▶ Scikit-learn often receives processed data from Pandas.

Key idea

Pandas connects real-world data files with Python-based data analysis.

Prerequisites and Environment Setup

Recommended background

- ▶ Basic Python: variables, lists, dictionaries, loops, and functions.
- ▶ Basic NumPy knowledge is helpful.
- ▶ A working environment: Jupyter Notebook, JupyterLab, Google Colab, VS Code, or similar.

Installation and import

```
pip install pandas
```

```
import pandas as pd  
print(pd.__version__)
```

Creating a DataFrame

```
import pandas as pd

data = {
    "Name": ["An", "Binh", "Chi"],
    "Age": [20, 21, 19],
    "GPA": [3.2, 3.6, 3.4]
}

df = pd.DataFrame(data)
print(df)
```

Interpretation

Each key in the dictionary becomes a column. Each list supplies the column values.

Code Output: First DataFrame

When printing df, Pandas displays a table with 3 rows and 3 columns.

	Name	Age	GPA
0	An	20	3.2
1	Binh	21	3.6
2	Chi	19	3.4

What to notice

The leftmost column 0, 1, 2 is the default index automatically created by Pandas.

DataFrame Structure

Component	Meaning
Rows	Records, observations, students, customers, products, or orders.
Columns	Variables, attributes, fields, or features.
Index	Row labels used for identification and selection.
Column labels	Names used to access variables.
Values	The actual data stored inside the table.

Example

A student table may contain StudentID, Name, Age, Major, and GPA.

Series and DataFrame from the First Example

One column is usually a Series

```
print(sales["Price"])
```

```
0    10
1    20
2     8
Name: Price, dtype: int64
```

Multiple columns form a DataFrame

```
print(sales[["Product", "Price"]])
```

	Product	Price
0	A	10
1	B	20
2	C	8

Remember

A labeled one-dimensional column is a Series; a two-dimensional table is a DataFrame.

How to Read Pandas Commands

Most Pandas commands follow one of these patterns:

`pd.function(input, option=value)` or `df.method(option=value)`

- ▶ `pd`: standard alias for the Pandas library.
- ▶ `df`: a variable usually used to store a DataFrame.
- ▶ `function`: a package-level function, such as `pd.read_csv()`.
- ▶ `method`: an operation attached to a DataFrame or Series.
- ▶ `option=value`: named arguments controlling command behavior.

Example

`pd.read_csv("sales.csv")` reads a CSV file into a DataFrame.

Command Pattern: Object, Attribute, Method, Function

Form	Meaning
<code>pd.DataFrame(data)</code>	Calls a Pandas function to create a table.
<code>df.shape</code>	Reads an attribute describing rows and columns.
<code>df.head()</code>	Calls a method attached to an existing DataFrame.
<code>pd.merge(left, right)</code>	Calls a Pandas function to combine two tables.
<code>df["Sales"].mean()</code>	Selects a column, then calls a Series method.

Teaching note

For every command, students should identify three parts: input, operation, and output.

Mini Example: Explain Each Line

```
import pandas as pd

sales = pd.DataFrame({
    "Product": ["A", "B", "C"],
    "Quantity": [2, 3, 5],
    "Price": [10, 20, 8]
})

sales["Revenue"] = sales["Quantity"] * sales["Price"]
print(sales)
```

- ▶ `pd.DataFrame(...)` creates a tabular dataset.
- ▶ Each dictionary key becomes a column name.
- ▶ `sales["Revenue"] = ...` creates a new calculated column.
- ▶ Multiplication is performed row by row across the selected columns.

Code Output: Revenue Example

After running the previous example, Revenue is created as $\text{Quantity} * \text{Price}$.

	Product	Quantity	Price	Revenue
0	A	2	10	20
1	B	3	20	60
2	C	5	8	40

Reading the result

Each row is one product. For example, product B has $\text{Quantity} = 3$, $\text{Price} = 20$, so $\text{Revenue} = 60$.

Quick Check: What Is Pandas?

Question 1. Which alias is conventionally used for Pandas?

- ▶ A. pn
- ▶ B. pd
- ▶ C. ps
- ▶ D. pa

Question 2. Which Pandas structure is two-dimensional?

- ▶ A. Series
- ▶ B. tuple
- ▶ C. DataFrame
- ▶ D. ndarray

Inspecting a DataFrame

```
print(df.head())  
print(df.tail())  
print(df.shape)  
print(df.columns)  
print(df.index)  
print(df.dtypes)  
df.info()  
print(df.describe())
```

Purpose

Inspection answers basic questions: How many rows? Which columns? Which data types? Are there missing values?

Example Output: Inspecting a DataFrame

For the student DataFrame above, some inspection commands return:

```
print(df.shape)
# (3, 3)

print(df.columns)
# Index(['Name', 'Age', 'GPA'], dtype='object')

print(df.dtypes)
# Name      object
# Age       int64
# GPA       float64
# dtype: object
```

Reading the output

shape = (3, 3) means the table has 3 rows and 3 columns. object is often used for string-like data.

Key Commands: Inspecting DataFrames

Command	Meaning
<code>df.head()</code>	Display the first rows.
<code>df.tail()</code>	Display the last rows.
<code>df.shape</code>	Return number of rows and columns.
<code>df.columns</code>	Return column labels.
<code>df.index</code>	Return row labels.
<code>df.dtypes</code>	Return the data type of each column.
<code>df.info()</code>	Structural summary of the DataFrame.
<code>df.describe()</code>	Descriptive statistics for numerical columns.

Mini Exercise: Create and Inspect Data

Create a DataFrame named `products` with columns `Product`, `Price`, and `Stock`.

```
products = pd.DataFrame({  
    "Product": ["Laptop", "Mouse", "Keyboard"],  
    "Price": [1200, 25, 45],  
    "Stock": [5, 50, 30]  
})  
  
print(products.head())  
print(products.shape)  
print(products.dtypes)
```

Check

All columns must contain the same number of values.

Code Output: Product DataFrame

When running the mini exercise, students can expect output like this:

	Product	Price	Stock
0	Laptop	1200	5
1	Mouse	25	50
2	Keyboard	45	30

```
print(products.shape)  
# (3, 3)
```

```
print(products.dtypes)  
# Product      object  
# Price        int64  
# Stock        int64  
# dtype: object
```

Quick Check: DataFrame Basics

Question 1. Which command creates a DataFrame?

- ▶ A. `pd.DataFrame()`
- ▶ B. `pd.SeriesFrame()`
- ▶ C. `np.DataFrame()`
- ▶ D. `df.create()`

Question 2. Which command returns the number of rows and columns?

- ▶ A. `df.shape`
- ▶ B. `df.mean()`
- ▶ C. `df.merge()`
- ▶ D. `df.plot()`

Creating a Series

A **Series** is a one-dimensional labeled array.

```
s = pd.Series([10, 20, 30], index=["A", "B", "C"])  
print(s)  
print(s.index)  
print(s.values)  
print(s.dtype)
```

Interpretation

A Series is like one labeled column. Pandas aligns Series by index labels during arithmetic operations.

Code Output: Series

```
A    10  
B    20  
C    30  
dtype: int64  
  
Index(['A', 'B', 'C'], dtype='object')  
[10 20 30]  
int64
```

Reading the output

The left column contains index labels; the right column contains Series values.

Series Alignment by Label

```
x = pd.Series([10, 20, 30], index=["A", "B", "C"])
y = pd.Series([1, 2, 3], index=["B", "C", "D"])

result = x + y
print(result)
```

Expected idea

Only matching labels are added directly. Unmatched labels produce missing values.

Code Output: Series Alignment

```
A      NaN  
B      21.0  
C      32.0  
D      NaN  
dtype: float64
```

Why NaN?

Labels B and C appear in both Series and are added. Label A appears only in x, and D appears only in y, so the result is NaN.

Example Data for Selection, Filtering, and Sorting

To make `loc`, `iloc`, filtering, and sorting examples clear, we use the following student table.

```
df = pd.DataFrame({
    "StudentID": ["S01", "S02", "S03", "S04"],
    "Name": ["An", "Binh", "Chi", "Dung"],
    "Age": [20, 21, 19, 22],
    "GPA": [3.2, 3.6, 3.4, 3.8]
})

print(df)
```

	StudentID	Name	Age	GPA
0	S01	An	20	3.2
1	S02	Binh	21	3.6
2	S03	Chi	19	3.4
3	S04	Dung	22	3.8

Selecting Columns

One column returns a Series

```
name = df["Name"]  
print(name)
```

Multiple columns return a DataFrame

```
subset = df[["Name", "GPA"]]  
print(subset)
```

Syntax note

Use one pair of brackets for one column; use a list of column names for multiple columns.

Code Output: Selecting Columns

One column: Series

```
0      An
1     Binh
2     Chi
3     Dung
Name: Name, dtype: object
```

Multiple columns: DataFrame

```
      Name  GPA
0      An  3.2
1     Binh  3.6
2     Chi  3.4
3     Dung  3.8
```

Selecting Rows with loc and iloc

Label-based selection

```
df = df.set_index("StudentID")  
print(df.loc["S01"])
```

Position-based selection

```
print(df.iloc[0])  
print(df.iloc[0:2])
```

Core distinction

loc uses labels; iloc uses integer positions.

Code Output: loc and iloc

After setting StudentID as the index, loc can select by student ID.

```
print(df.loc["S01"])  
# Name      An  
# Age       20  
# GPA       3.2  
# Name: S01, dtype: object  
  
print(df.iloc[0:2])  
#           Name  Age  GPA  
# StudentID  
# S01         An   20   3.2  
# S02        Binh   21   3.6
```

Selecting Rows and Columns Together

```
# label-based rows and columns
selected = df.loc[["S01", "S02"], ["Name", "GPA"]]
print(selected)

# position-based rows and columns
selected = df.iloc[0:2, [0, 2]]
print(selected)
```

Reading the syntax

The part before the comma selects rows; the part after the comma selects columns.

Key Commands: Series and Data Selection

Command	Meaning
<code>pd.Series(data)</code>	Create a one-dimensional labeled array.
<code>s.index</code>	Return Series labels.
<code>s.values</code>	Return underlying values.
<code>df["col"]</code>	Select one column as a Series.
<code>df[["c1","c2"]]</code>	Select multiple columns as a DataFrame.
<code>df.loc[label]</code>	Select by row label.
<code>df.iloc[position]</code>	Select by integer position.

Quick Check: Series and Selection

Question 1. A Pandas Series is:

- ▶ A. a two-dimensional table
- ▶ B. a one-dimensional labeled array
- ▶ C. a database server
- ▶ D. a plotting package

Question 2. Which selector is label-based?

- ▶ A. `iloc`
- ▶ B. `loc`
- ▶ C. `shape`
- ▶ D. `head`

Boolean Filtering

Filter rows that satisfy a condition.

```
high_gpa = df[df["GPA"] >= 3.5]  
print(high_gpa)
```

Interpretation

`df["GPA"] >= 3.5` creates a Boolean Series. Pandas keeps rows where the value is True.

Code Output: Filtering Data

With the condition `GPA >= 3.5`, Pandas keeps only matching rows.

```
high_gpa = df[df["GPA"] >= 3.5]  
print(high_gpa)
```

```
#           Name  Age  GPA  
# StudentID  
# S02      Binh   21  3.6  
# S04      Dung   22  3.8
```

Reading the output

Students S02 and S04 have GPA greater than or equal to 3.5, so they appear in the result.

Filtering with Multiple Conditions

```
selected = df[
    (df["Age"] >= 20) &
    (df["GPA"] >= 3.5)
]
print(selected)
```

- ▶ Use & for AND.
- ▶ Use | for OR.
- ▶ Use ~ for NOT.
- ▶ Put each condition inside parentheses.

Sorting DataFrames

```
# sort by one column
by_gpa = df.sort_values("GPA")

# descending order
by_gpa_desc = df.sort_values("GPA", ascending=False)

# sort by multiple columns
ordered = df.sort_values(
    ["Age", "GPA"],
    ascending=[True, False]
)
```

Business example

Sort customers by revenue descending, then by order frequency descending.

Code Output: Multiple Conditions and Sorting

```
selected = df[(df["Age"] >= 20) & (df["GPA"] >= 3.5)]  
print(selected)
```

```
#           Name  Age  GPA  
# StudentID  
# S02      Binh   21  3.6  
# S04      Dung   22  3.8
```

```
print(df.sort_values("GPA", ascending=False))
```

```
#           Name  Age  GPA  
# StudentID  
# S04      Dung   22  3.8  
# S02      Binh   21  3.6  
# S03       Chi   19  3.4  
# S01       An    20  3.2
```

Key Commands: Filtering and Sorting

Command	Meaning
<code>df[df["x"] > a]</code>	Keep rows where column x exceeds a.
<code>(cond1) & (cond2)</code>	Combine conditions with AND.
<code>(cond1) (cond2)</code>	Combine conditions with OR.
<code>~cond</code>	Negate a condition.
<code>df.sort_values("x")</code>	Sort rows by one column.
<code>ascending=False</code>	Sort in descending order.

Quick Check: Filtering and Sorting

Question 1. Which expression filters rows with GPA ≥ 3.5 ?

- ▶ A. `df[df["GPA"]  $\geq$  3.5]`
- ▶ B. `df.sort_values("GPA")`
- ▶ C. `df.read_csv("GPA")`
- ▶ D. `df.merge("GPA")`

Question 2. Why should parentheses be used when combining multiple filter conditions?

Reading and Writing CSV Files

Read CSV

```
df = pd.read_csv("data.csv")

df = pd.read_csv(
    "data.csv",
    sep=",",
    header=0,
    encoding="utf-8"
)
```

Write CSV

```
df.to_csv(
    "output.csv",
    index=False
)
```

Important

`index=False` prevents the DataFrame index from being exported as an extra column.

Reading and Writing Excel Files

Read Excel

```
df = pd.read_excel(  
    "data.xlsx",  
    sheet_name="Sheet1"  
)
```

Write Excel

```
df.to_excel(  
    "output.xlsx",  
    index=False  
)
```

Teaching note

Excel is common in business contexts, but students should still inspect types and missing values after importing.

JSON and Text Files

JSON

```
df = pd.read_json("data.json")

df.to_json(
    "output.json",
    orient="records"
)
```

Delimited text file

```
df = pd.read_csv(
    "data.txt",
    sep="\t"
)
```

Idea

Many structured text files can be read using `read_csv()` by changing the separator.

Key Commands: Data I/O

Command	Role
<code>pd.read_csv()</code>	Read CSV or delimited text files.
<code>df.to_csv()</code>	Write data to a CSV file.
<code>pd.read_excel()</code>	Read Excel files.
<code>df.to_excel()</code>	Write data to an Excel file.
<code>pd.read_json()</code>	Read JSON files.
<code>df.to_json()</code>	Write data to a JSON file.

Quick Check: Data I/O

Question 1. Which function reads a CSV file?

- ▶ A. `pd.read_csv()`
- ▶ B. `pd.open_csv()`
- ▶ C. `pd.load_table_only()`
- ▶ D. `df.csv_read()`

Question 2. Which argument commonly prevents the index from being exported?

- ▶ A. `index=False`
- ▶ B. `index=True`
- ▶ C. `header=None`
- ▶ D. `drop_index=True`

Why Data Cleaning Matters

Real-world datasets may contain:

- ▶ missing values;
- ▶ duplicate rows;
- ▶ inconsistent data types;
- ▶ empty or irrelevant columns;
- ▶ inconsistent text, extra spaces, or inconsistent capitalization;
- ▶ numeric data mixed with text.

Goal

Data cleaning improves accuracy, consistency, and usability before analysis or modeling.

Example of Messy Data and Inspection Output

```
df = pd.DataFrame({  
    "Name": [" An ", "Binh", "Chi", "Chi"],  
    "Age": [20, None, 22, 22],  
    "Price": ["10", "20", "unknown", "30"]  
})  
  
print(df)  
print(df.isna().sum())
```

	Name	Age	Price
0	An	20.0	10
1	Binh	NaN	20
2	Chi	22.0	unknown
3	Chi	22.0	30

```
Name      0  
Age        1  
Price      0  
dtype: int64
```


Detect Missing Values

```
print(df.isna())  
print(df.isna().sum())  
  
missing_by_column = df.isna().sum()  
total_missing = df.isna().sum().sum()
```

Interpretation

`isna()` identifies missing cells; `sum()` counts missing values by column or across the whole DataFrame.

Remove or Fill Missing Values

Remove missing values

```
rows_complete = df.dropna()  
cols_complete = df.dropna(axis=1)
```

Fill missing values

```
df["Age"] = df["Age"].fillna(  
    df["Age"].mean()  
)  
  
df["City"] = df["City"].fillna("Unknown")
```

Decision question

Should missing values be removed, imputed, or explicitly labeled? The answer depends on the analysis goal.

Duplicates and Data Types

Duplicate rows

```
duplicate_mask = df.duplicated()
duplicate_count = duplicate_mask.sum()
df = df.drop_duplicates()
```

Data types

```
print(df.dtypes)

df["Quantity"] = df["Quantity"].astype(int)

df["Price"] = pd.to_numeric(
    df["Price"],
    errors="coerce"
)
```

String Cleaning

Text columns often need standardization.

```
df["Name"] = df["Name"].str.strip()
df["Name"] = df["Name"].str.lower()

df["City"] = df["City"].str.replace(
    "HN",
    "Hanoi"
)
```

Common issue

" An ", "AN", and "an" may refer to the same name, but they are different strings unless cleaned.

Code Output: Data Cleaning

```
df["Age"] = df["Age"].fillna(df["Age"].mean())  
df["Name"] = df["Name"].str.strip().str.lower()  
df["Price"] = pd.to_numeric(df["Price"], errors="coerce")  
print(df)
```

	Name	Age	Price
0	an	20.00	10.0
1	binh	21.33	20.0
2	chi	22.00	NaN
3	chi	22.00	30.0

Reading the output

unknown is converted to NaN; names are stripped and converted to lowercase.

Key Commands: Data Cleaning

Command	Meaning
<code>df.isna()</code>	Detect missing values.
<code>df.isna().sum()</code>	Count missing values.
<code>df.dropna()</code>	Remove observations with missing data.
<code>df.fillna(value)</code>	Replace missing values.
<code>df.duplicated()</code>	Detect duplicate rows.
<code>df.drop_duplicates()</code>	Remove duplicate rows.
<code>df.astype(type)</code>	Convert data type.
<code>pd.to_numeric()</code>	Convert values to numeric form.
<code>Series.str.*</code>	Apply string-processing methods.

Quick Check: Data Cleaning

Question 1. Which method removes rows containing missing values?

- ▶ A. `dropna()`
- ▶ B. `fillna()`
- ▶ C. `duplicated()`
- ▶ D. `astype()`

Question 2. Which method removes duplicate rows?

- ▶ A. `drop_duplicates()`
- ▶ B. `dropna()`
- ▶ C. `sort_values()`
- ▶ D. `merge()`

Example Data for Operations and Aggregation

```
df = pd.DataFrame({  
    "Region": ["North", "North", "South", "South", "East"],  
    "Product": ["A", "B", "A", "B", "A"],  
    "Quantity": [2, 3, 5, 2, 1],  
    "Price": [10, 20, 8, 30, 12]  
})  
print(df)
```

	Region	Product	Quantity	Price
0	North	A	2	10
1	North	B	3	20
2	South	A	5	8
3	South	B	2	30
4	East	A	1	12

Creating Calculated Columns

```
df["Revenue"] = df["Quantity"] * df["Price"]  
  
total_revenue = df["Revenue"].sum()  
print(df)  
print(total_revenue)
```

Business example

Revenue, profit margin, discount rate, customer lifetime value, and delivery delay can often be created from existing columns.

Code Output: Calculated Column

	Region	Product	Quantity	Price	Revenue
0	North	A	2	10	20
1	North	B	3	20	60
2	South	A	5	8	40
3	South	B	2	30	60
4	East	A	1	12	12

192

Reading the output

The total revenue is the sum of all row-level revenue values: $20 + 60 + 40 + 60 + 12 = 192$.

Applying Functions

Use `map()`

```
df["Status"] = df["Score"].map(  
    lambda x: "Pass" if x >= 5 else "Fail"  
)
```

Use `apply()`

```
df["Squared"] = df["Value"].apply(  
    lambda x: x ** 2  
)
```

Good practice

Prefer vectorized column operations when possible; use `apply()` when custom logic is needed.

Normalization and Z-score Standardization

```
# Min-max normalization
df["Normalized"] = (
    (df["Value"] - df["Value"].min()) /
    (df["Value"].max() - df["Value"].min())
)

# Z-score standardization
df["Z"] = (
    (df["Value"] - df["Value"].mean()) /
    df["Value"].std()
)
```

Interpretation

Normalization rescales values to a fixed range; standardization expresses values in standard-deviation units.

Descriptive Statistics

```
print(df.describe())

print(df["Revenue"].mean())
print(df["Revenue"].median())
print(df["Revenue"].min())
print(df["Revenue"].max())
print(df["Revenue"].std())
```

Analytical role

Descriptive statistics help detect scale, central tendency, variation, and possible outliers.

Grouping with `groupby()`

```
region_total = df.groupby("Region")["Revenue"].sum()
print(region_total)

region_stats = (
    df.groupby("Region")["Revenue"]
      .agg(["count", "sum", "mean", "min", "max"])
)
print(region_stats)
```

Split-apply-combine

Pandas splits rows into groups, applies aggregation to each group, and combines the results.

Code Output: groupby and agg

```
Region
East      12
North     80
South    100
Name: Revenue, dtype: int64
```

	count	sum	mean	min	max
Region					
East	1	12	12.0	12	12
North	2	80	40.0	20	60
South	2	100	50.0	40	60

Reading the output

South has the highest total revenue. The second table gives several summary statistics per region.

Group by Multiple Columns

```
summary = (  
    df.groupby(["Region", "Product"])["Revenue"]  
        .sum()  
)  
print(summary)
```

Business interpretation

This can answer questions such as: Which product contributes the most revenue in each region?

Key Commands: Operations and Aggregation

Command	Meaning
<code>df["new"] = ...</code>	Create or transform a column.
<code>Series.map()</code>	Map values to new values.
<code>Series.apply()</code>	Apply a function to Series elements.
<code>df.describe()</code>	Descriptive statistics.
<code>df.groupby()</code>	Group observations.
<code>.agg()</code>	Apply multiple aggregation functions.
<code>mean(), median(), std()</code>	Common numerical summaries.

Quick Check: Operations

Question 1. Which method groups observations by categories?

- ▶ A. `groupby()`
- ▶ B. `dropna()`
- ▶ C. `astype()`
- ▶ D. `sort_index()`

Question 2. Complete the idea: Revenue = Quantity ... Price.

Concatenating DataFrames with concat

```
combined = pd.concat(  
    [df1, df2],  
    axis=0  
)
```

- ▶ `axis=0`: stack rows vertically.
- ▶ `axis=1`: combine columns side by side.

Example

Concatenate January sales and February sales when both tables have the same columns.

Example Data for merge

```
customers = pd.DataFrame({
    "CustomerID": ["C01", "C02", "C03"],
    "Name": ["An", "Binh", "Chi"]
})

orders = pd.DataFrame({
    "OrderID": ["001", "002", "003"],
    "CustomerID": ["C01", "C02", "C02"],
    "Amount": [100, 250, 300]
})
```

Idea

CustomerID is the common key used to combine the two tables.

Merging Tables with merge

```
result = pd.merge(
    customers,
    orders,
    on="CustomerID",
    how="inner"
)
print(result)
```

- ▶ inner: keep matching records only.
- ▶ left: keep all records from the left table.
- ▶ right: keep all records from the right table.
- ▶ outer: keep all records from both tables.

Code Output: merge

	CustomerID	Name	OrderID	Amount
0	C01	An	001	100
1	C02	Binh	002	250
2	C02	Binh	003	300

Reading the output

Customer C02 appears twice because C02 has two orders. Customer C03 is absent in an inner merge because there is no matching order.

Pivot Tables

A pivot table summarizes numerical values across categories.

```
pivot = pd.pivot_table(  
    df,  
    values="Revenue",  
    index="Region",  
    columns="Product",  
    aggfunc="sum"  
)  
print(pivot)
```

Interpretation

Rows are regions, columns are products, and cells contain total revenue.

Code Output: pivot table

For the example revenue data, a pivot table may look like this:

Product Region	A	B
East	12.0	NaN
North	20.0	60.0
South	40.0	60.0

Reading the output

The cell at row South, column B equals 60, meaning total revenue of product B in South is 60.

Reshaping Data: pivot and melt

Long to wide

```
wide = df.pivot(  
    index="Date",  
    columns="Product",  
    values="Sales"  
)
```

Wide to long

```
long = pd.melt(  
    wide.reset_index(),  
    id_vars="Date"  
)
```

Idea

The data shape should match the analytical task: visualization, aggregation, modeling, or reporting.

Key Commands: Combine and Reshape

Command	Meaning
<code>pd.concat()</code>	Combine DataFrames along an axis.
<code>pd.merge()</code>	Combine DataFrames using key columns.
<code>how="inner"</code>	Keep matching records only.
<code>how="left"</code>	Keep all records from the left table.
<code>df.pivot()</code>	Reshape long data to wide format.
<code>pd.melt()</code>	Reshape wide data to long format.
<code>pd.pivot_table()</code>	Create a summarized table.

Quick Check: Combine and Reshape

Question 1. Which function combines DataFrames using a key column?

- ▶ A. `pd.merge()`
- ▶ B. `pd.mean()`
- ▶ C. `pd.reshape()`
- ▶ D. `pd.filter_rows()`

Question 2. What does a pivot table do?

Correlation

Correlation measures association between numerical variables.

```
print(df.corr(numeric_only=True))  
  
print(  
    df["Advertising"].corr(df["Sales"])  
)
```

Important

Correlation does not by itself imply causation.

Code Output: Correlation and Time Series

```
print(df[["Revenue", "Quantity", "Price"]].corr())
```

	Revenue	Quantity	Price
Revenue	1.000000	0.620174	0.594089
Quantity	0.620174	1.000000	-0.272772
Price	0.594089	-0.272772	1.000000

Reading the output

The correlation matrix is symmetric. Values closer to 1 indicate a stronger positive linear relationship; however, correlation does not prove causation.

Quick Visualization with Pandas

Pandas plotting functions are built on top of Matplotlib.

```
df.plot(x="Month", y="Sales", kind="line")  
df.plot(x="Product", y="Sales", kind="bar")
```

```
df["Sales"].plot(kind="hist")  
df.plot(x="Advertising", y="Sales", kind="scatter")
```

Time-Series Pipeline

```
df["Date"] = pd.to_datetime(df["Date"])  
df = df.set_index("Date")  
df = df.sort_index()  
  
weekly = df["Sales"].resample("W").sum()  
monthly = df["Sales"].resample("ME").sum()
```

Time-series idea

After converting the date column and setting it as the index, Pandas can aggregate data by calendar periods.

Datetime Components and Moving Average

Date components

```
df["Year"] = df.index.year  
df["Month"] = df.index.month  
df["Day"] = df.index.day
```

Moving average

```
df["MovingAvg"] = (  
    df["Sales"]  
        .rolling(window=3)  
        .mean()  
)
```


Key Commands: Advanced Operations

Command	Meaning
<code>df.corr()</code>	Calculate correlation matrix.
<code>Series.corr()</code>	Calculate correlation between two Series.
<code>df.plot()</code>	Create quick visualizations.
<code>pd.to_datetime()</code>	Convert values to datetime.
<code>df.set_index()</code>	Set a column as the index.
<code>df.resample()</code>	Aggregate time-series data by time intervals.
<code>Series.rolling()</code>	Calculate rolling-window statistics.

Quick Check: Advanced Operations

Question 1. Which function converts a column to datetime?

- ▶ A. `pd.to_datetime()`
- ▶ B. `pd.date_convert_only()`
- ▶ C. `df.datetime()`
- ▶ D. `pd.time_series()`

Question 2. Which method calculates a correlation matrix?

- ▶ A. `corr()`
- ▶ B. `merge()`
- ▶ C. `dropna()`
- ▶ D. `pivot()`

Content Summary I

Topic	Main idea
Pandas	Python library for structured data manipulation and analysis.
Series	One-dimensional labeled array.
DataFrame	Two-dimensional labeled table.
Inspection	Use <code>head()</code> , <code>shape</code> , <code>info()</code> , and <code>describe()</code> .
Selection	Use column brackets, <code>loc</code> , and <code>iloc</code> .
Filtering	Use Boolean conditions to keep relevant rows.

Content Summary II

Topic	Main idea
Data I/O	Read and write CSV, Excel, JSON, and text files.
Cleaning	Handle missing values, duplicates, data types, and strings.
Operations	Create calculated columns and transform data.
Aggregation	Use <code>groupby()</code> and <code>agg()</code> for summary reports.
Combination	Use <code>concat()</code> and <code>merge()</code> for multiple tables.
Advanced	Correlation, visualization, and time-series operations.

Practice: Student Data

Exercise 1

Create a DataFrame with at least 10 students containing:

- ▶ StudentID, Name, Age, Major, GPA.

Perform:

- 1 inspection with `head()`, `shape`, `info()`, and `describe()`;
- 2 filtering for `GPA >= 3.5`;
- 3 sorting by GPA descending;
- 4 selecting rows using `loc` and `iloc`.

Practice: Missing Data and Cleaning

Exercise 2

Add missing values to the student dataset. Then:

- 1 count missing values;
- 2 fill missing Age with mean Age;
- 3 fill missing GPA with median GPA;
- 4 remove rows with missing Name.

Exercise 3

Create a messy dataset containing spaces in names, duplicate rows, and mixed numeric/text values. Clean it using `str.strip()`, `drop_duplicates()`, and `pd.to_numeric()`.

Practice: Sales Analysis

Exercise 4

Create a sales dataset containing Date, Region, Product, Quantity, and Price.
Create:

$$\text{Revenue} = \text{Quantity} \times \text{Price}.$$

Then calculate:

- 1 total revenue;
- 2 revenue by Region;
- 3 revenue by Product;
- 4 average revenue by Region;
- 5 a Region $\times$ Product pivot table.

Practice: Merge and Time Series

Exercise 5: Merge

Create `customers`(`CustomerID`, `Name`, `City`) and `orders`(`OrderID`, `CustomerID`, `Amount`). Perform inner merge, left merge, and identify customers without orders.

Exercise 6: Time series

Create daily sales data for at least 30 days. Convert `Date` to `datetime`, set it as index, calculate weekly total sales, calculate a 7-day moving average, and create a line plot.

Final Reflection

- ▶ What is the difference between a Series and a DataFrame?
- ▶ When should we use `loc`, and when should we use `iloc`?
- ▶ Why should we inspect shape, dtypes, and missing values early?
- ▶ What is the difference between `concat()` and `merge()`?
- ▶ Why is `groupby()` important in business analytics?
- ▶ Why does correlation not automatically prove causation?

Suggested Answers: Quick Checks

Section	Answers
What is Pandas?	B; C
DataFrame basics	A; A
Series and selection	B; B
Data I/O	A; A
Data cleaning	A; A
Operations	A; Revenue = Quantity $\times$ Price
Combine and reshape	A; summarizes values across categories
Advanced operations	A; A